Delivery ETA Prediction + Delay Risk

1. Problem Framing and Assumptions

The goal of this project is to improve delivery reliability for a logistics platform by solving two problems:

A - ETA Prediction (Regression)

Predict the delivery time in minutes for each order.

This helps:

- show accurate delivery time to customers
- plan routes better
- reduce delays

B - Delay Risk (Classification)

Predict whether a delivery will be late.

To define "late", I used:

Late if Delivery_Time > 75th percentile of training data

Reason:

- fixed threshold (like 60 min) may not fit all cities
- percentile adapts automatically to dataset
- top 25% slow deliveries are treated as late
- gives balanced classes for training

This makes the classification realistic and data-driven.

2. Data Split Strategy and Leakage Prevention (Time-based split**)**

I sorted deliveries by date and time and used:

first 80% → training
last 20% → testing

Reason:

- deliveries happen over time
- model should learn from past and predict future
- random split may leak future information

This simulates real-world deployment.

Leakage prevention

I removed columns that contain future or useless information:

- Pickup_Time (happens after order → leakage)
- Order_ID (no predictive value)
- raw datetime columns

This ensures the model only uses information available at prediction time.

3. Feature Engineering (important part)

I created meaningful features instead of using raw data.

Time features

- hour of day
- day of week
- weekend flag

Reason: traffic and delivery speed change with time.

Distance feature

- Haversine distance between store and drop location

Reason: longer distance → more time.

Categorical encoding

- weather, traffic, vehicle, area, category

Reason: these conditions directly affect delivery speed.

These features help the model learn real-world patterns.

4. Model Comparison and Metrics

A – Regression (ETA)

(I used XGBoost because it includes built-in regularization. Model complexity was controlled using a limited tree depth and a small learning rate, along with default L1/L2 penalties. This helped prevent overfitting, as seen from the small gap between training and validation curves).

Model	MAE
Baseline(mean)	41.7 min
XGBoost	17.3 min

RMSE = 22.3 min (The RMSE of 22.3 minutes indicates that most predictions are within roughly 20–22 minutes of the actual delivery time, with larger errors being penalized more heavily)

Improvement:

- error reduced by ~58%
- model predicts much closer to actual time

B – Classification (Delay Risk)

Metric	Value
ROC-AUC	0.96
F1-score	0.81
Baseline F1	0.00

Interpretation:

- high ROC-AUC → good separation of late vs on-time
- good F1 → balanced precision and recall
- much better than baseline

5. Error Analysis and Insights

For the regression task, the model significantly improves over the baseline, reducing MAE from 41.7 minutes to 17.3 minutes. The training and validation curves remain close, indicating good generalization with no major overfitting. Residual analysis shows small errors for short and medium deliveries, while larger errors occur for long-distance orders, which is expected due to higher uncertainty from traffic and weather.

Feature importance shows that traffic conditions, delivery distance, order category, and agent rating contribute the most to predictions. These factors logically affect delivery time, confirming that the model learns meaningful operational patterns rather than noise.

For the classification task, the delay model achieves a ROC-AUC of 0.96 and F1-score of 0.81, showing strong performance in identifying late deliveries. Delays are mostly associated with heavy traffic, bad weather, and longer routes. Overall, both models perform reliably and are suitable for practical use.

6. Training vs Validation Curves

The training and validation curves decrease together and stay close throughout training. The small gap indicates that the model is not overfitting. Both models show stable convergence during learning. Therefore, they are expected to generalize well to unseen future deliveries.

7. Deployment and Monitoring

The trained models would be deployed as a REST API service that receives order details and returns the predicted ETA and delay probability in real time. The API can be integrated with the delivery platform to show customers accurate ETAs and flag high-risk deliveries. Model performance would be monitored daily using metrics such as MAE for ETA and F1/ROC-AUC for delay prediction. Prediction logs would be stored to compare actual vs predicted times. Data drift in traffic, weather, or order patterns would also be tracked. If performance drops, the model would be retrained periodically with recent data to maintain accuracy.